Data Structures & Algorithms
Assignment 3: Graphs

due on Thursday 26 March 2026
1pm GMT

▶ **Problem 1.** Given an undirected unweighted graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges, the diameter of the graph G is defined as the largest shortest distance between any pair of vertices. More formally,

$$\mathsf{Diam}(G) = \max_{(u,v)\in V\times V} \mathsf{Dist}(u, v),$$

where $\mathsf{Dist}(u, v)$ is the shortest distance between the vertices u and v.

Write a Java code that outputs an integer T such that $\mathsf{Diam}(G) \leq T \leq 2 \times \mathsf{Diam}(G)$. The graph will be represented using an adjacency list. Note that if two vertices u and v are adjacent, then v appears in the list of neighbours of u, and u also appears in the list of neighbours of v. For example, the graph drawn in Fig. 1 will be represented on input as:

```
// Graph with 5 vertices (0 to 4) and edges: (0,1), (1,2), (2,3), (3,4), (0, 2)
int[][] G = {{1, 2},    // Neighbors of vertex 0
             {0, 2},    // Neighbors of vertex 1
             {0, 1, 3}, // Neighbors of vertex 2
             {2, 4},    // Neighbors of vertex 3
             {3}};      // Neighbors of vertex 4
```

The exact diameter of this graph is 3 since $\mathsf{Dist}(v_0, v_4) = 3$. Since the output T must satisfy $\mathsf{Diam}(G) \leq T \leq 2 \times \mathsf{Diam}(G)$, a valid output could be `int approxDiameter(G) = 5` (or any integer from 3 to 6).

All inputs will be connected graphs. Your code should also work on the empty graph (i.e., the graph with no vertices). Find as efficient an implementation as you can. (6 points)

Rules: Make sure that you follow all the following rules:

1. Implement all necessary data structures and algorithms. **Do not use any Java libraries!** In particular, `java.util`, and functions contained therein, are not allowed to be used in this assignment.
2. Submit your solution in **a single java file** containing a class `Assignment3` with a method `approxDiameter` with the following header:

```
public class Assignment3 {
    public static int approxDiameter(int[][] graph) {
        // your solution here
    }
}
```

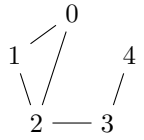


Figure 1: Example graph

Your file will be renamed to `Assignment3.java` before compiling.

3. **Make sure you file compiles!** If your code does not compile, you will receive no credit. Do not include your name in the file, and comment out all unnecessary text.

Your solution will be tested by an automated script on several inputs of larger and larger sizes. We will mark both correctness and efficiency, with 2 points awarded for correct code, and the remaining 4 points awarded depending on the execution time of your code. **To receive full marks, make sure your code is as efficient as possible and passes all the test cases.**

Test your solution using the code provided in file `TestAssignment3.java`. Note that the file contains some, but not all, test cases your code will be evaluated on. The total number of test cases, and their sizes, will be much larger.

This assignment consist of one problem worth **6 points** which is 6% of module credit.